

IMPLEMENTACJA ALGORYTMU KLASTERYZACJI

CHAMELEON

hierarchical clustering using dynamic modeling

Projekt referencyjnej, scikit-learn-kompatybilnej implementacji algorytmu Karypisa, Hana i Kumara (1999) jako pakietu PyPI `pychameleon`.

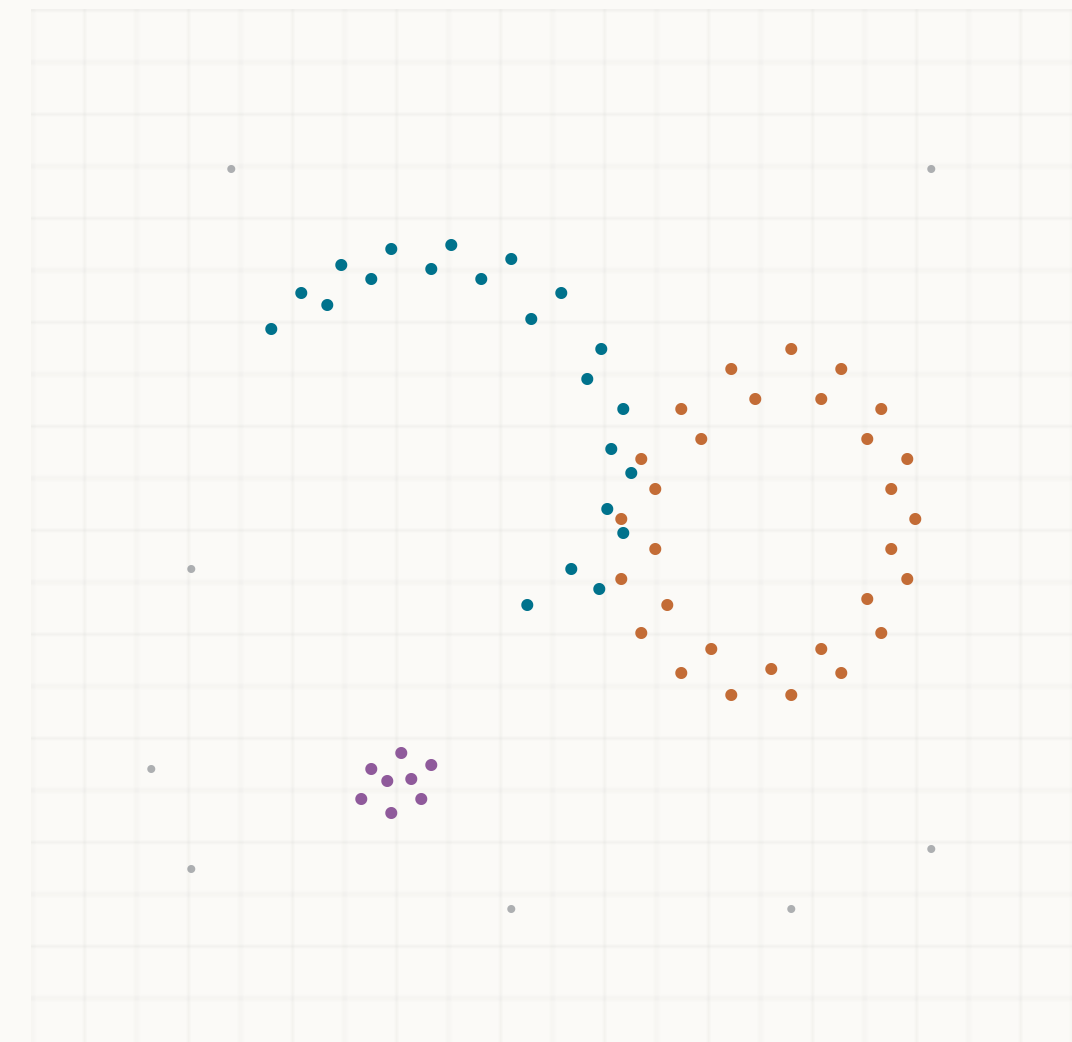


fig. 01 · DS3-like 2D dataset · arbitrary shape, varying density

PLAN PREZENTACJI

Plan prezentacji

zakres · technologie · testy

– KONTEKST

- 01.** Problem i dlaczego CHAMELEON
- 02.** Algorytm — fazy 0 / I / II
- 03.** Miary RI · RC · merge_score

– CZĘŚĆ 02 · TECHNOLOGIE

- 07.** Stos technologiczny
- 08.** Decyzje projektowe i uzasadnienia
- 09.** Złożoność vs implementacja referencyjna

– CZĘŚĆ 01 · ZAKRES PROJEKTU

- 04.** Co implementujemy — public API
- 05.** Architektura modułów
- 06.** Deliverables Etap 1 / Etap 2

– CZĘŚĆ 03 · PLANY TESTÓW

- 10.** 5-warstwowa strategia walidacji
- 11.** Zbiory danych testowych
- 12.** Metryki, progi, ryzyka

PROBLEM

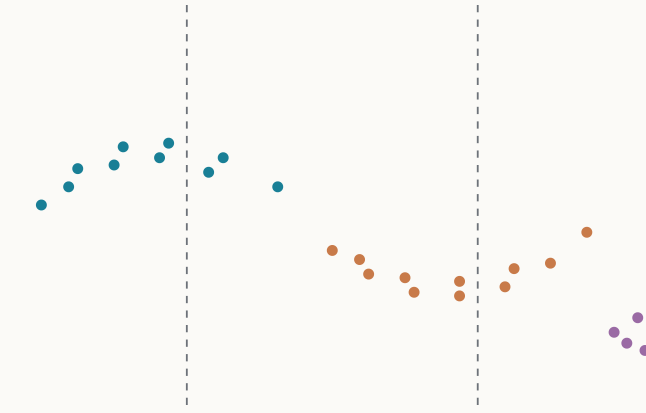
Klasyczne algorytmy zawodzą, gdy klastry mają różne kształty, gęstości i rozmiary.

CURE, ROCK, single/complete/average linkage, k-means, DBSCAN — używają statycznego modelu klastra: stały próg podobieństwa, zbiór reprezentantów lub macierz dystansów. Tracą skuteczność, gdy w jednym zbiorze sąsiadują klaster gęsty i rozproszony.

— ROZWIĄZANIE CHAMELEON-A

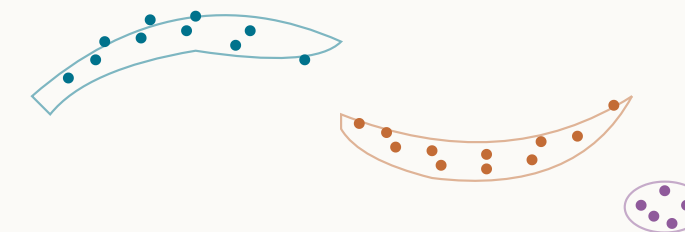
- **Dynamiczne modelowanie** — kryterium łączenia uwzględnia *wewnętrzną* charakterystykę dwóch kandydatów (gęstość, interconnectivity), nie globalny próg.
- **Graf k-NN** jako reprezentacja danych — naturalnie modeluje lokalną strukturę gęstości.
- **Dwie fazy:** szybka bisekcja → ostrożna aglomeracja sterowana relacjami względnymi (RI · RC).

— K-MEANS (K=3)



Sferyczne centroidy rozcinają wydłużone klastry. ARI ≈ 0.4 .

— CHAMELEON



Klastry zachowują kształt. ARI > 0.95 na DS1–DS5.

ALGORYTM

Trzy fazy. Jedna kolejka decyzji.



– WEJŚCIE

$X \in \mathbb{R}^{n \times d}$ · macierz cech, np.float64

– PARAMETRY KLUCZOWE

n_clusters · k_nn · min_cluster_size · α

– WYJŚCIE

labels_ $\in \mathbb{Z}^n$ · etykiety w $[0, n_clusters)$

DYNAMICZNE MODELOWANIE

Łączymy parę, której relacja przypomina ich własną **strukturę wewnętrzną**.

– RELATIVE INTERCONNECTIVITY (EQ. 1)

$$RI(C_i, C_j) = \frac{|EC_{ij}|}{(|EC_i| + |EC_j|) / 2}$$

Stosunek krawędzi przeciętych granicą do średniego bisektora obu klastrów.

– RELATIVE CLOSENESS (EQ. 2)

$$RC(C_i, C_j) = \frac{\bar{S}_{EC_{ij}}}{w_i \cdot \bar{S}_{EC_i} + w_j \cdot \bar{S}_{EC_j}}$$

Stosunek średniej wagi krawędzi granicznych do ważonej średniej bisektorów.

– MERGE SCORE (EQ. 4)

$$score(C_i, C_j) = RI \cdot RC^\alpha$$

$\alpha > 1$ — preferuje closeness. $\alpha < 1$ — preferuje interconnectivity. Domyślnie $\alpha = 2.0$.

– INTUICJA

Klaster „rozproszony” ma niski wewnętrzny RI; klaster „gęsty” ma wysoki. Norm. *względna* sprawia, że oba mogą zostać zachowane lub sklejone z odpowiednim partnerem — bez globalnego progu.

– OZNACZENIA

EC_{C_i, C_j}	krawędzie krzyżujące granicę między C_i a C_j
EC_{C_i}	bisektor klastra C_i — krawędzie 2-way min-cut
$\bar{S}_E$	średnia waga krawędzi w zbiorze E
w_i	$ C_i / (C_i + C_j)$ — waga proporcji

01

CZĘŚĆ 01

Zakres projektu

Co dokładnie planujemy zaimplementować, w jakim zakresie, jakie cele realizuje projekt, jakie są deliverables Etapu 1 i Etapu 2.

CO DOKŁADNIE IMPLEMENTUJEMY

Pakiet `pychameleon` — jedna klasa, scikit-learn API.

— PUBLIC API

```
# jedyny publiczny import
from pychameleon import Chameleon

model = Chameleon(
    n_clusters=8,
    k_nn=10,
    min_cluster_size=0.025,
    alpha=2.0,
)
labels = model.fit_predict(X)

# sklearn-compatible attributes
model.labels_      # ndarray (n,)
model.n_clusters_
model.n_features_in_
```

— CO WCHODZI W ZAKRES

- Pełna implementacja faz **0 / I / II** zgodna z paperem Karypis 1999, §4.
- **scikit-learn estimator** — `BaseEstimator` + `ClusterMixin`, `check_estimator` pass.
- **Optymalizacje:** `KDTree` zamiast $O(n^2)$, lazy heap, cache bisekcji wewnętrznych.
- **Publikacja PyPI** jako `pychameleon v0.1` (lato 2026).

— POZA ZAKRESEM (ROADMAP V0.2 / V0.3)

obsługa szumu (`min_samples_per_cluster`) • parametr `random_state` • backend ANN dla $d > 10$ (Annoy / HNSW) • GPU

ARCHITEKTURA

Pięć modułów, każdy mapuje się na sekcję paperu.

MODUŁ	SEKCJA PAPERU	GŁÓWNE FUNKCJE / KLASY	ZŁOŻONOŚĆ
graph.py	§4.2	knn_graph(X, k) → (adjacency, edge_weights) · KDTree-backed	$O(n \log n)$
partition.py	§4.4 Phase I	initial_subclusters(adj, weights, min_size) → labels	$O(n \log^2 n)$
metrics.py	§4.3	relative_interconnectivity · relative_closeness · merge_score	vectorised
merger.py	§4.4 Phase II	merge_to_k_clusters(adj, weights, init_labels, k, α) → labels · heapq + lazy	$O(m^2 \log m)$
chameleon.py	§4 całość	class Chameleon(ClusterMixin, BaseEstimator) · public	orkiestracja
_types.py	—	aliasy: FloatMatrix · Labels · AdjacencyList · EdgeWeights	—

— LAYOUT

src-layout (PEP 660) · zapobiega bug-om import-order;
testy odseparowane.

— STYL

mypy --strict + ruff · zgodność ze sklearn-style libs
(HDBSCAN, scikit-learn-extra).

— WEJŚCIE

np.ndarray (n, d) float64 · walidacja przez
sklearn.utils._validate_data.

DELIVERABLES

Etap 1 — projekt i szkielec. Etap 2 — implementacja i walidacja.

— ETAP 1 deadline 08.05.2026 • 15 / 45 pkt

- 01 **Dokumentacja projektu** (PDF, 8 sekcji) • upload na LEON.
- 02 **Prezentacja konsultacyjna** • niniejszy deck.
- 03 **Szkielet repozytorium** • pyproject, src-layout, CI, pre-commit.
- 04 **Skeleton modułów** • sygnatury, type hints, docstringi.
- 05 **Plan testów** • 5 warstw, fixtures, zbiory vendored.
- 06 **Benchmark referencji** • uruchomienie Moonpuck na 3 zbiorach.

— ETAP 2 deadline koniec semestru • 30 / 45 pkt

- 01 **Pełna implementacja** • graph • partition • metrics • merger • Chameleon.
- 02 **Testy jednostkowe** • ~25 testów • pokrycie ≥ 90%.
- 03 **Testy integracyjne i porównawcze** • vs Moonpuck • ARI > 0.9 na 3 zbiorach.
- 04 **Testy parametryczne i skalowalności** — sweep parametrów k_nn, α, min_cluster_size.
- 05 **Sprawozdanie końcowe** • wyniki, porównania, roadmap v0.2.
- 06 **Publikacja** • pychameleon v0.1 na PyPI (post-deadline).

02

CZĘŚĆ 02

Wybrane technologie

Stos technologiczny — język programowania, biblioteki, narzędzia do testowania, build i CI/CD; uzasadnienie wyborów.

STOS TECHNOLOGICZNY

Python 3.11+ · numpy · scipy · pymetis · scikit-learn.

– RUNTIME / CORE

python ≥ 3.11
PEP 695, structural pattern matching

numpy ≥ 1.24
macierze, ufuncs, vectorisation

scipy ≥ 1.11
KDTree, sparse matrices

scikit-learn ≥ 1.3
BaseEstimator, ClusterMixin, validation, ARI

pymetis ≥ 2023.1
multilevel min-cut Karypis–Kumar 1998

– JAKOŚĆ KODU

ruff
lint + format · 10× szybszy od
pylint+flake8+isort razem

mypy --strict
statyczna kontrola typów; standard
sklearn-style

pre-commit
git hooks: ruff, mypy, end-of-file-fixer

conventional commits
historia czytelna dla recenzenta

– TESTY

pytest ≥ 7.4
runner, fixtures, parametryzacja

pytest-cov
pokrycie linii — target ≥ 90%

numpy.testing
tolerancyjne porównania floating-point

sklearn.utils.estimator_checks
~30 sanity checks for any sklearn estimator

adjusted_rand_score
metryka oceny vs ground truth

– BUILD · CI · VIZ

uv
resolver + venv · drop-in pip replacement

hatchling
build backend · minimalny pyproject

matplotlib · seaborn
extra [viz] · benchmarki, debug plots

DECYZJE PROJEKTOWE

Każdy wybór ma uzasadnienie — algorytmiczne lub ekosystemowe.

DECYZJA	WYBÓR	UZASADNIENIE
k-NN backend	<code>scipy.spatial.KDTree</code>	$O(n \log n)$ zamiast $O(n^2)$ w naiwnej impl. referencyjnej • ~47× speedup dla $n = 8000$ • zależność i tak wnoszona przez sklearn.
Reprezentacja grafu	<code>list[NDArray[int64]]</code>	direct CSR compatibility z pymetis • iteracja per-wierzchołek dominuje w Fazie II • szybsze niż networkx.
Bisekcja Phase I + EC	<code>pymetis.part_graph(2, ...)</code>	multilevel KL refinement • stan-sztuki dla min-cutu • referencyjny w paperze Karypis 1999.
Kolejka Phase II	<code>heapq + lazy invalidation</code>	amortyzowany $O(\log m)$ push/pop • zero zewnętrznych zależności • eliminuje $O(m^2)$ rebuild po każdym scaleniu.
API wejścia	<code>X (n × d, float64)</code>	konwencja sklearn • pipeline-compatible • automatyczna walidacja przez <code>_validate_data</code> .
Layout pakietu	<code>src-layout</code>	zapobiega importom z working dir (PEP 660) • standard sklearn / numpy / scipy.
Build backend	<code>hatchling</code>	minimalny, nowoczesny pyproject.toml • brak setup.py • wymagany przez PyPI publish flow.
Type checker	<code>mypy --strict</code>	standard sklearn-style libs • public API stabilny od v0.1 • długoterminowa pielęgnowalność.

ZŁOŻONOŚĆ I EFEKTYWNOŚĆ

Dominuje $O(n \log^2 n)$. Spodziewany speedup vs implementacja referencyjna — 50–100x.

– ZŁOŻONOŚĆ PER FAZA

FAZA	PYCHAMELEON	MOONPUCK (REF.)
Faza 0 • k-NN graph	$O(n \log n)$ KDTree	$O(n^2)$ naiwna macierz
Faza I • bisekcja	$O(n \log^2 n)$	$O(n \log^2 n)$
Faza II • aglomeracja	$O(m^2 \log m)$ lazy	$O(m^3)$ rebuild
Pamięć	$O(nd + nk + m^2)$	$O(n^2)$

– BENCHMARK MOONPUCK (REF.) NA MBP M1

ZBIÓR	N	K	K_NN	RUNTIME	BOTTLENECK
aggregation	788	7	20	9.95 s	balanced
smileface	644	4	10	2.08 s	balanced
t4_8k	8 000	6	20	674.47 s	naiwny $O(n^2)$ k-NN

N = LICZBA PUNKTÓW • M ≈ N/20 SUB-KLASTRÓW

– ŹRÓDŁA SPEEDUPU

- 01KDTree • ~47x dla n = 8000.
- 02Cache bisekcji |EC_{C_i}|.
- 03Lazy invalidation heap'a.
- 04numpy ufuncs w metrics.py • stała 10–50x.

~47x

K-NN SPEEDUP
(N = 8000)

50–100x

TOTAL
SPEEDUP TARGET

– OGRANICZENIA

KDTree degeneruje dla d > 10 (roadmap: Annoy/HNSW). pymetis nie jest deterministyczny — w v0.2 wprowadzimy random_state.

03

CZĘŚĆ 03

Plany testów

Strategia weryfikacji poprawności i jakości implementacji — jednostkowe, integracyjne, porównawcze, parametryczne; metryki oceny i zbiory danych.

STRATEGIA 5-WARSTWOWA

Każda warstwa odpowiada na inne pytanie. Pokrycie $\geq 90\%$ linii.

WARSTWA	CO WERYFIKUJE	LICZBA / PRÓG	STATUS (26.04)
Jednostkowe	każdy moduł osobno · hand-computed wartości referencyjne dla małych grafów	~25 (5 mod × ~5)	SZKIELET
Integracyjne (e2e)	fit() end-to-end na 3 datasetach; ARI vs ground truth	7 testów · ARI > 0.85	12 SKELETON PASS
sklearn-compat	check_estimator(Chameleon()) — pełen sanity sweep estymatora	~30 wewnętrznych	PO FIT()
Porównawcze	wyniki vs Moonpuck na tych samych parametrach	3 datasety · ARI > 0.9	ETAP 2
Parametryczne	sweep k_nn · α · min_cluster_size	3 sweep × ~5 wart.	ETAP 2
Skalowalności	n vs runtime · slope log-log ≈ 1.1	6 punktów n ∈ [100, 50k]	ETAP 2

— METRYKA GŁÓWNA

adjusted_rand_score — niezmiennicza na permutację etykiet, kluczowa dla klasteryzacji bez nadzoru.

— PRÓG ARI > 0.9, NIE 1.0

pymetis nie jest deterministyczny (KL refinement losowy). Walidujemy zachowanie, nie identyczność labels.

— POKRYCIE

pytest --cov $\geq 90\%$ linii · raport term-missing w CI.

ZBIORY DANYCH TESTOWYCH

Pięć zbiorów. Trzy z paperu / literatury, dwa generowane.

ZBIÓR	N	DIM	K	CHARAKTERYSTYKA
Aggregation Gionis 2007	788	2	7	klastry różnych rozmiarów, łączące się · ground truth ✓
smileface Moonpuck repo	644	2	4	klastry niewypukłe — łuk uśmiechu · ground truth ✓
t4_8k DS3 · paper §5	8 000	2	6	klastry z dziurami i szumem · referencyjny w paperze
small_blobs	60	2	3	deterministyczna fixture · sanity check · seed=42
make_blobs	var	2	5	sklearn-generated · skalowalność $n \in [100, 50k]$

Zbiory referencyjne *vendored* w `tests/data/`; ładowane przez fixtures w `tests/conftest.py`. Pochodzenie udokumentowane w `docs/07_zbiory_danych.md`.

— PYTANIA I DYSKUSJA

Dziękuję.

— RYZYKA I MITIGACJE

- **Niedeterminizm pymetis** — KL refinement losowy. Mitigacja: próg ARI > 0.9, nie 1.0; `random_state` w v0.2.
- **Wysokie wymiary** — KDTree degeneruje dla $d > 10$. Mitigacja: zakres testów $d = 2$; ANN backend w v0.3.
- **Brak obsługi szumu** — paper Karypis 1999 przypisuje wszystkie punkty. Mitigacja: `min_samples_per_cluster` w v0.2.

— BIBLIOGRAFIA KLUCZOWA

- [1] Karypis, Han, Kumar · *CHAMELEON* · IEEE Computer 32(8) 1999.
- [2] Karypis, Kumar · *Multilevel partitioning* · SIAM J. Sci. Comput. 1998.
- [5] Gionis, Mannila, Tsaparas · *Clustering aggregation* · ACM TKDD 2007.
- [6] Pedregosa et al. · *scikit-learn* · JMLR 2011.

Oliwier Szypczyn
github.com/oszypczy/pychameleon